Team learning guide · October 18, 2026

The HPC Assistant, explained from zero

How we teach a small AI to help people use supercomputers, written so a complete beginner can follow it, then going deeper step by step.

How to read this guide. Every topic has three layers:

- **Simple version**: the idea as a story, no jargon.
- **A bit deeper**: the real words experts use.
- **In our project**: the actual numbers and choices we made.

Read only the green boxes on a first pass. Come back for the blue and orange ones. The purple **Advanced corner** at the end has the maths.

1 The big idea in one story

SIMPLE VERSION

Imagine a very wise professor who knows a lot about supercomputers. The professor is huge: they live in a big building and need lots of electricity. You can't take them home.

So the professor teaches a clever young student. The student practises tens of thousands of questions with the professor's answers. Afterwards the student is small enough to live in your laptop, and helps you any time: no internet, no cost, and your questions stay private.

Our project builds that student. The professor is a big AI model, the student is a small AI model, and the lessons are questions and answers (Q&A).

Documents → Big teacher writes Q&A → Checker verifies → Small student learns → Judge scores → Runs on your laptop

IN OUR PROJECT

Teacher: Qwen3-32B (32 billion "knobs"). **Student**: Qwen3-4B (4 billion). **Checker**: Gemma 4 26B. **Judge**: gpt-oss-20b. The goal comes from `GOAL.md`: a laptop assistant that beats the original small model.

2 What is an AI language model?

SIMPLE VERSION

It's a super-powered "guess the next word" machine. If you type "The cat sat on the…", it guesses "mat". It has read so much text that its guesses form whole helpful answers.

A BIT DEEPER

- **Token**: a piece of a word, roughly ¾ of a word. Models read and write tokens.
- **Parameters** (the "B" in 4B): the numbers inside the model that store what it learned. More parameters usually means smarter, but bigger and slower.
- **Prompt**: everything we give the model before it answers: instructions, the question, maybe documents.
- **Hallucination**: when the model confidently makes something up. Our main enemy.

3 What is HPC, and why do people need help?

SIMPLE VERSION

A supercomputer is thousands of powerful computers joined together. Scientists use them for weather, medicine and AI. You don't use it directly: you write a small "job card" saying what you need (how many machines, how long) and put it in a **queue**, like a waiting list at a busy restaurant. A program called **Slurm** is the waiter who seats each job when space frees up.

Writing job cards is tricky, and every supercomputer has its own rules. That's where our assistant helps.

A BIT DEEPER

- **HPC** = High-Performance Computing. **Node** = one computer in the cluster. **GPU** = the chip that does AI maths fast.
- **Job script**: a text file starting with lines like `#SBATCH --gres=gpu:4` (ask for 4 GPUs).
- **Partition**: a named section of the machine, like `boost_usr_prod` on Leonardo or `standard-g` on LUMI.
- Every site sets Slurm up differently. That's why mixing sites up is dangerous (see chapter 14).

IN OUR PROJECT

We train on **Leonardo** (CINECA, Italy): each node has 4 NVIDIA A100 GPUs with 64 GB each. Our fast "debug" queue allows jobs of at most 30 minutes, 2 jobs at a time, 32 GPUs in total. Our access ends October 27.

4 Why a small model on a laptop?

SIMPLE VERSION

Big models live on rented servers: you pay, you need internet, and your questions leave your computer. A small model on your laptop is free, works offline and keeps everything private. The trick is making the small one nearly as good as the big one, for one subject.

A BIT DEEPER

This is called **distillation**: a big "teacher" model's knowledge is poured into a small "student" model. We use **sequence-level distillation**: the teacher writes full answers and the student learns to write the same kind of answers. This works even when teacher and student are different model families.

5 Making the lessons: questions and answers

SIMPLE VERSION

We collected the official help pages of Europe's supercomputer centres. For each page, the teacher writes a realistic question a user might ask, then a careful answer that uses only that page. One page can become a few different questions.

A BIT DEEPER

- **Source grounding**: every answer is tied to its source page, so we can check it.
- **Question types**: how-to, debugging, concepts, scripts. Some answers also practise **tool calls** (asking a tool for live job status) and **ask-back** (asking the user for a missing detail instead of guessing).
- **Personas**: questions come from different kinds of users (beginner, sysadmin, researcher), so the student sees variety.

IN OUR PROJECT

We downloaded **39,496** pages from all European AI Factories, EuroHPC and CINECA (including the two links our mentor Simeon sent), plus HPC example code repositories. Batch 1 planned 90,060 questions; the top-up batch another 52,500 with new question angles; a coverage batch asks only about chunks the first two never used.

6 Checking the lessons (verification)

SIMPLE VERSION

Even good teachers make mistakes. So a second, different AI reads every answer next to its source page and asks: "Is every claim here really supported by the page?" Then simple computer checks look for broken commands and fake options. Anything doubtful is thrown away. We'd rather have fewer lessons than wrong ones.

A BIT DEEPER

- **Independent reviewer**: a different model family from the teacher, so they don't share blind spots.
- **Deterministic checks**: shell-syntax check of scripts, known Slurm flags only.
- **Decontamination**: questions too similar to the test questions are removed, otherwise the test would be cheating.
- **Near-duplicate removal**: questions that are almost the same as an earlier one are dropped.

IN OUR PROJECT

Only about 30% of generated answers pass all checks, which shows how strict the checks are. The most common reason was the reviewer calling a claim "supported" without pointing to the evidence; those rows are held back, not released. As of today we have **about 42,800 verified Q&A**, heading for 60K and then 100K+.

7 Is more Q&A always better?

SIMPLE VERSION

More *good* practice makes a better student. But copying the same page 100 times doesn't teach more, and practising wrong answers teaches mistakes. So: more is better when it's **correct, new and on-topic**.

A BIT DEEPER

- **Coverage**: new topics and sites help the most. That's why the coverage batch targets documents nobody asked about yet.
- **Diminishing returns**: each extra 10K rows usually helps a little less than the previous 10K.
- **Noise hurts**: a small share of wrong answers can noticeably lower scores. That's why verification is strict.
- We never assume; each round is **measured** against the previous one in the same judge run.

8 Open-book exams: RAG and RAFT

SIMPLE VERSION

A closed-book exam means answering from memory. An open-book exam lets you look things up. **RAG** gives the model an open book: before answering, we search the documents and paste the best 3 pieces into the question. **RAFT** is practising for open-book exams: during training, most lessons include the book pages, so the student learns to read them, use the right facts and ignore the wrong pieces.

A BIT DEEPER

- **RAG** = Retrieval-Augmented Generation. **Retriever** = the search engine. Ours uses **BM25**, a classic keyword-scoring method, inside SQLite (a small database file).
- **Chunk**: documents are cut into pieces of about 2,200 characters so the search returns focused parts.
- **RAFT** = Retrieval-Augmented Fine-Tuning: about 80% of training questions come with retrieved pages, about 20% without, so the model can do both.

IN OUR PROJECT

The built-in index has 7,159 AI Factory documents (48,061 chunks) plus generic Slurm, LUMI and CINECA pages. Search takes about **6 milliseconds**. The real cost is that 3 pages add about 1,900 tokens to the prompt, which a laptop has to read before answering.

9 Teaching the student: fine-tuning and LoRA

SIMPLE VERSION

Re-writing a whole textbook is expensive. Instead, you add sticky notes to the pages that matter. **LoRA** does exactly that: the big model stays frozen, and we train a small set of "sticky notes" (an **adapter**) on top. The adapter is about 130 MB instead of 8 GB.

A BIT DEEPER

- **Fine-tuning**: continuing training of an existing model on our own data.
- **LoRA** = Low-Rank Adaptation. Settings: rank 16, alpha 32 (how big and how strong the sticky notes are).
- **Learning rate** 5e-5: how big each learning step is. **Epoch**: one full pass over all lessons (we do 1).
- **Loss**: how wrong the student's guesses are; it goes down as it learns. Round 3 ended around 0.41.

10 The volume knob: adapter strength (×1.0, ×0.5, ×0.2)

SIMPLE VERSION

Think of the sticky notes as a volume knob. ×1.0 is full volume: the student follows its new lessons completely. ×0.5 is half volume: half new lessons, half original knowledge. Sometimes half volume sounds best, because the original model also knew useful things.

IN OUR PROJECT

In round 3, **×0.5 with documents** scored best. We test all three strengths every round and pick the winner by measurement.

11 Rolling dice: seeds, and a "soup" of students

SIMPLE VERSION

Training involves some randomness, like shuffling cards. A **seed** fixes the shuffle. We train 3 students with 3 different shuffles (seeds 42, 43, 44) to check results aren't luck. A **soup** blends the 3 students into one; blends are often steadier than any single one.

A BIT DEEPER

Averaging LoRA adapters naively is wrong, because each seed started from different random sticky notes. We stack the three adapters into one bigger adapter (rank 48), which equals their average change exactly. We checked: the error is about 0.0000001 of the size of the values.

12 How we score: the judge

SIMPLE VERSION

Like a cooking contest: two plates, a judge tastes both and picks the better one, or says "tie". We put our student's answer next to the original small model's answer (with no documents) and ask a judge AI which is better. Then we swap the plates and ask again, so the judge can't favour "whichever came first".

Win rate: win = 1 point, tie = ½ point, loss = 0. If both models were equally good, the score would be **50%**. Above 50% means our student is better.

A BIT DEEPER

- **Pairwise judging** in both orders; the judge was calibrated at 94% accuracy on known cases.
- **Test sets**: v3 (200 questions), v2 (66), d1 (106, a harder European development set). None are used in training.
- **Judge drift**: the same answers can score a few points differently in another judge run. So we only compare rounds **inside the same run**.
- **Paired bootstrap**: a statistics trick that resamples the questions thousands of times to see whether a gain is real or luck.

13 Why 80–90% is so hard

SIMPLE VERSION

A student rarely beats their teacher. Our teacher scores about 69% on v2 in this contest. And the opponent (the original small model) is already decent, so many questions end in a tie, which only counts half. 80–90% would mean beating it on almost every question.

A BIT DEEPER

Two honest ways to raise the ceiling:

1. **A stronger teacher**: Qwen3-235B, about 7 times bigger. We first measure its score before regenerating data with it.
2. **Best-of-n**: the teacher writes 2–4 answers per question and an independent reviewer keeps the best, so the lessons beat the teacher's average. The reviewer must not be the exam judge, or we'd be training to please the judge rather than to be correct.

14 Same commands, different sites: avoiding confusion

SIMPLE VERSION

Every supercomputer names its sections differently. If the student learns "use the gpu partition" without knowing which machine that was for, it will say it everywhere, which is wrong. So every fact must stay attached to its site name.

IN OUR PROJECT

Questions are written to name the site ("On LUMI, …"). Every document piece carries a `Site:` label. In round 5, only questions that name a specific system are trained without the document; vague ones keep it. In our laptop test without documents, the model wrongly said Leonardo uses a "gpu" partition. With documents it correctly said `boost_usr_prod`. That's proof both safeguards matter.

15 Licences: who allows what

SIMPLE VERSION

Web pages have owners. Some say "use freely, just credit us" (CC-BY), some say "not for commercial use" (NC), many say nothing. We tag every lesson with its page's licence. A public release only includes the freely usable part.

16 Running it all on the supercomputer, unattended

SIMPLE VERSION

Our queue only allows 30-minute jobs, but training takes hours. So the work is cut into small pieces that can stop and continue, like saving a video game. Little robot managers (**controllers**) start the next piece whenever one finishes, and a **watchdog** restarts any manager that stops by accident.

A BIT DEEPER

- **Shards**: the 90K questions are split into 200 parts, each processed separately and resumable.
- **Checkpoints**: training saves progress every 50 steps, so a 30-minute limit loses little.
- Lessons we learned the hard way: a crashed tool can take every manager down with it, and a temporary hiccup can look like a finished job. Both are now guarded against.

17 Shrinking it for the laptop: quantization

SIMPLE VERSION

A photo can be saved in a smaller file with slightly less detail. **Quantization** does that for models: each number is stored with fewer bits. The 8 GB model becomes a 2.5 GB file that a laptop can run, with only a small quality loss.

A BIT DEEPER

- **GGUF**: the file format used by **llama.cpp** and **Ollama**, the programs that run models on laptops.
- **Q4_K_M**: about 4 bits per number, a popular balance of size and quality.
- **Speed has two parts**: reading the prompt (prompt processing) and writing the answer (generation). Long prompts with documents make the first part slow.

IN OUR PROJECT

We built the 2.5 GB Q4_K_M file from the round-3 winner and tested it on 4 CPU threads. It works end to end. The speed in that test was low because of a generic, unoptimised build; a real laptop measurement comes next.

18 The assistant you actually use

SIMPLE VERSION

You type a question; it searches its built-in library, reads the best pages, and answers with sources. You can give it your own cluster's manual, and it'll read that first. It remembers the conversation, and it can decide to look something up again.

```
hpc-assistant "How do I request 2 GPUs for 4 hours with Slurm?"
hpc-assistant --add our-cluster-manual.pdf
hpc-assistant            # chat: /add FILE, /docs on|off, /reset, /quit
```

A BIT DEEPER

- **Built-in retrieval**: the same search used in testing, so users get what we measured.
- **Tool calling**: the model can ask for the `search_docs` tool by writing a structured request; the assistant runs the search and gives back the results.
- **Generic fallback**: if the site-specific search finds nothing, it searches every site.
- **Bigger vision** (from Paritosh): the same recipe for any subject (food, budget, diet planning). Bring your own data, get an expert assistant.

19 Where we are now

Round	Training data	Result / status
3	19,411 verified Q&A	×0.5 + docs: v3 **0.682**, v2 **0.635**, d1 **0.554**, versus the original model with docs at 0.640 / 0.545 / 0.493. Better on all three sets.
4	~44K (adds all-AI-Factory batch 1)	Training starts this afternoon; scores tonight. Adds the seed soup.
5	60K+ (adds top-up and coverage batches)	Also learns site-anchored facts without documents; scores Monday.
Next	100K+, maybe a stronger teacher	Measure the 235B teacher first; best-of-n if worthwhile.

The original Hugging Face model card scores closed-book: v3 0.528, v2 0.481. Round 3 without documents: v3 0.570, v2 0.504.

20 Advanced corner

LORA IN ONE LINE

A layer's weights W get an update $\Delta W = (\alpha/r)\cdot B\cdot A$, where A is r×in and B is out×r with r = 16. Only A and B are trained. Strength ×s multiplies B by s. A soup of n seeds is $B_{soup} = [B_1 \ldots B_n]\cdot s/n$ and $A_{soup} = [A_1; \ldots; A_n]$, so $\Delta W_{soup} = (s/n)\cdot \Sigma\ \Delta W_i$ exactly, with rank n·r and α scaled by n so α/r stays the same.

WIN RATE AND PAIRED BOOTSTRAP

Per question: score = (result in order 1 + result in order 2)/2, with win 1, tie ½, loss 0. Win rate = mean over questions. To compare two arms judged in the same run, take the per-question differences d_i, resample them with replacement 10,000 times, and read the 2.5% and 97.5% points of the resampled means. If the interval stays above 0, the gain is unlikely to be luck.

BM25 RETRIEVAL

Each chunk gets a score that rises when it contains the question's rarer words, with diminishing returns for repeats and a correction for long chunks. Fast, transparent, no neural network needed, and it runs inside SQLite's FTS5 search engine.

NEAR-DUPLICATES

Each question becomes a set of overlapping word groups (shingles). Two questions are duplicates when their Jaccard similarity (shared ÷ total) is at least 0.8. An inverted index finds candidates fast, without comparing all pairs.

WHY TIES PULL SCORES TOWARD 50%

If our student wins 60% of questions, ties 30% and loses 10%, the win rate is 0.60 + 0.15 = 0.75. To reach 0.90 you'd need roughly 85% wins with only a few ties. Ties are common when both answers are correct, which is why very high win rates are rare.

Check yourself. (1) Why do we swap the plates for the judge? (2) Why can a ×0.5 adapter beat ×1.0? (3) Why is "more data" only good when it's verified? (4) What is the difference between RAG and RAFT? (5) Why must site names stay attached to facts?

Glossary

Adapter	The small trained "sticky notes" (LoRA) added to a frozen model.
Base model	The original model before our training (Qwen3-4B).
Chunk	A piece of a document (~2,200 characters) used for search.
Closed-book / with docs	Answering from memory, or with retrieved pages in the prompt.
Distillation	Teaching a small model using a big model's answers.
GGUF / Q4_K_M	Laptop model file format / 4-bit compression level.
Judge	The AI that compares two answers and picks the better one.
Node / GPU	One computer in a supercomputer / the chip that does AI maths.
RAG / RAFT	Search-then-answer / training with search results included.
Seed	A number that fixes the randomness of training.
Slurm	The queue manager that runs jobs on a supercomputer.
Token	A piece of a word; the unit models read and write.
Verified Q&A	A question and answer that passed the reviewer and all automatic checks.
Win rate	Share of comparisons won (ties count half) against the base model.